

Path Finder Game

Shayan.....DT-22037

Ahmed Hassan.....DT-22041

Neha Laeeq.....DT-22002

Talal Khan.....DT-22043

INSTRUCTOR:.....Faiza Nasim

Table of Contents:

1. Introduction:	3
2. Tools and Techniques Used:	3
3. Flow Diagram:	4
4. Main Algorithms Implemented in the Projects:	5
5. Output:	7

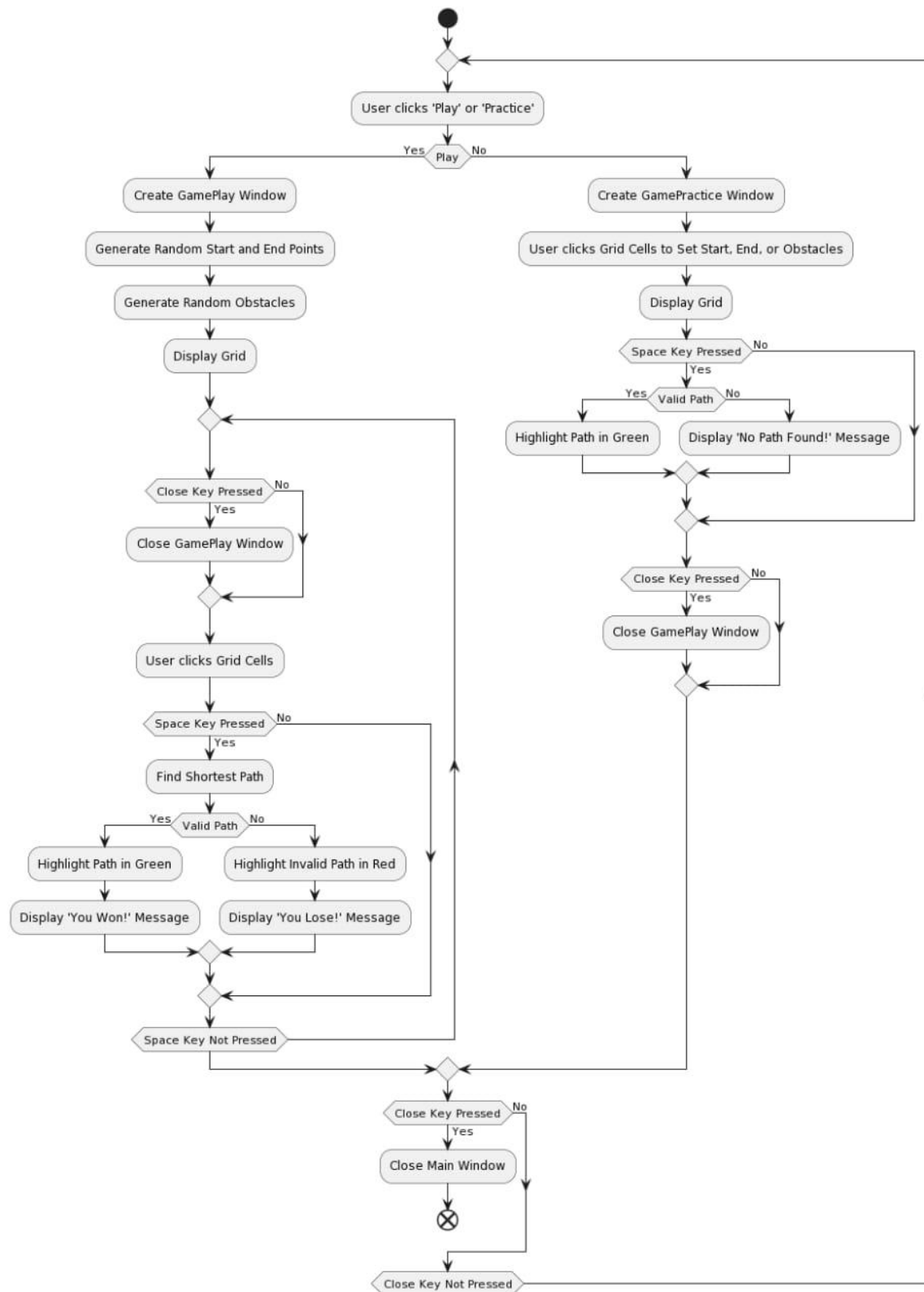
1. Introduction:

The Python code creates an interactive path-finding game using the *A* algorithm* and the *Tkinter library*. The game aims to engage users in navigating through a grid-based environment, avoiding obstacles, and finding the shortest path from a starting point to an endpoint. The code includes features for playing and practicing path creation.

2. Tools and Techniques Used:

- a) Programming Language:** Python
- b) Graphical User Interface (GUI) Library:** Tkinter
- c) Algorithm:** A* (A-star) algorithm for efficient pathfinding
- d) Data Structures:** Priority Queue for managing exploration priorities (used by A* algorithm).

3. Flow Diagram:



4. Main Algorithms Implemented in the Projects:

a) Pathfinding Algorithm:

- The "astar" function implements the A* pathfinding algorithm. It uses a priority queue to explore nodes based on their estimated total cost (heuristic + actual cost).
- The algorithm calculates the cost-so-far and the heuristic for each node and updates the priority queue accordingly.
- The heuristic function provides an estimate of the distance between two points.

b) PriorityQueue Class:

- The PriorityQueue class is a simple implementation of a priority queue using a list.
- It supports putting elements into the queue, getting the element with the highest priority, and checking if the queue is empty.

c) Grid-Based Movement:

- The movement in the grid is defined by the `are_cells_adjacent` function, which checks if two cells are adjacent in the grid.
- The obstacles are represented as cells that cannot be traversed.

d) GamePlay Class:

- Manages the GUI for the game where the player can play the pathfinding challenge.
- Uses the A* algorithm to find the shortest path from the start to the end point, considering obstacles.
- Handles user interaction by allowing the marking of cells and checking if the path is valid.
- The user starts from the "start" grid and is asked to find the shortest path to the "end" grid. The grids selected by the user turn "blue". Upon pressing spacebar the program checks the path and compares it to the shortest path found using the A* pathfinding algorithm.
- The user is given a total of three tries. If the user fails to find the correct path within the try limit then the game finishes and the answer is shown.

e) GamePractice Class:

- Allows users to practice creating their own start and end points along with obstacles.
- Utilizes the A* algorithm to find and display the shortest path between the user-defined start and end points, considering the obstacles.

f) Main Class:

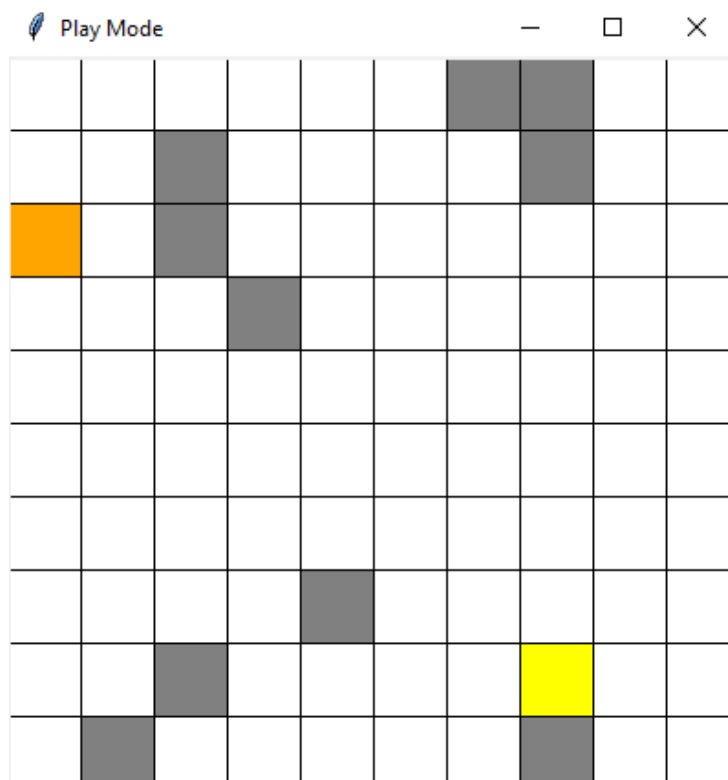
- Sets up the main GUI window and provides options to play the game or practice.
- Implements the A* pathfinding algorithm through the GamePlay and GamePractice classes.

5. Output:

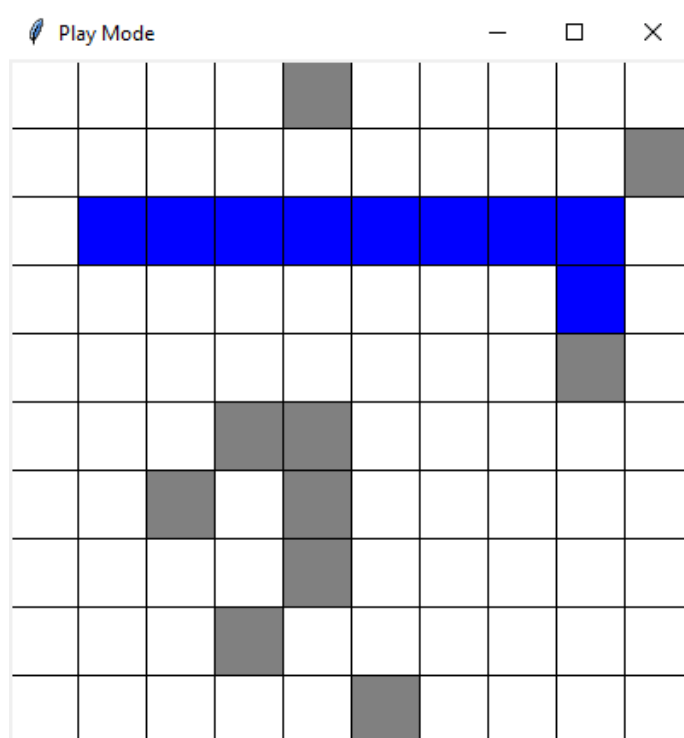
Upon code execution, the user is presented with this window.



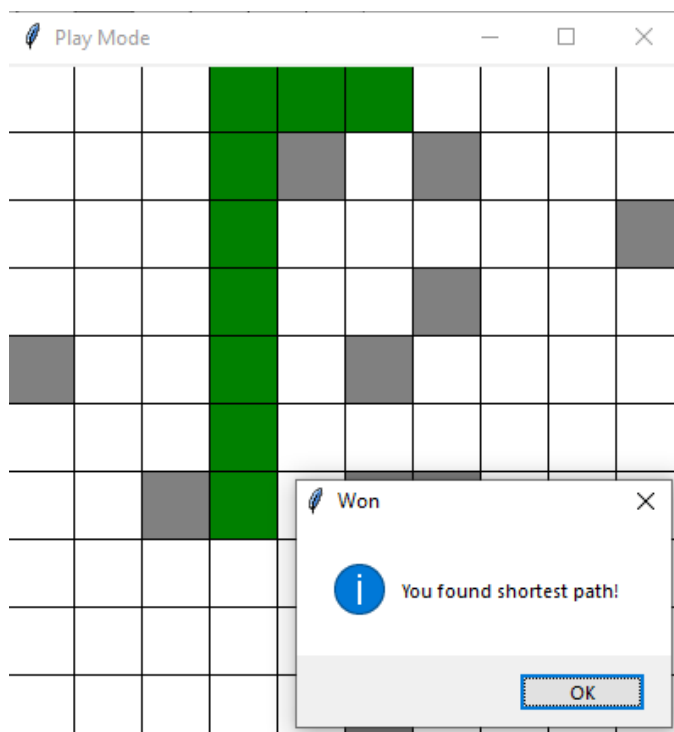
Upon selecting play mode, the user is presented with the following grid:



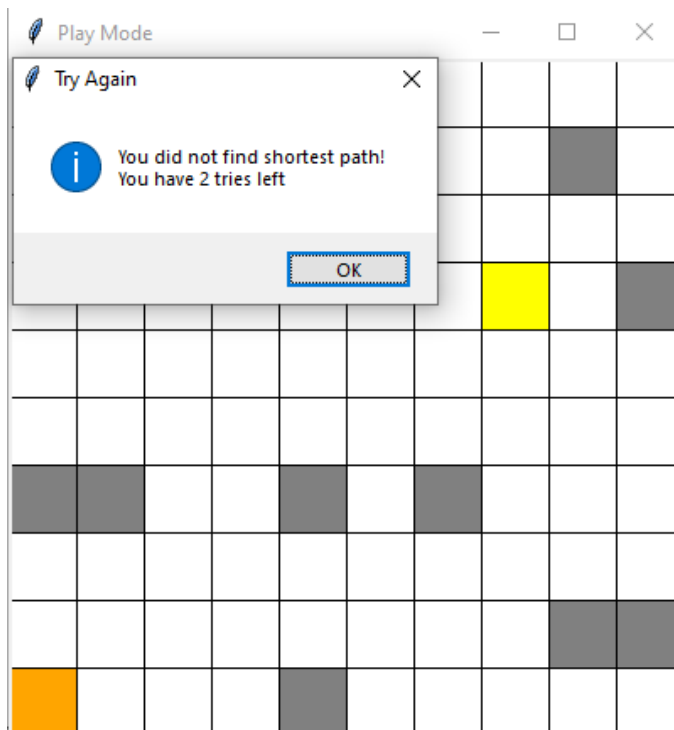
The user selects the shorts path from start to end point:



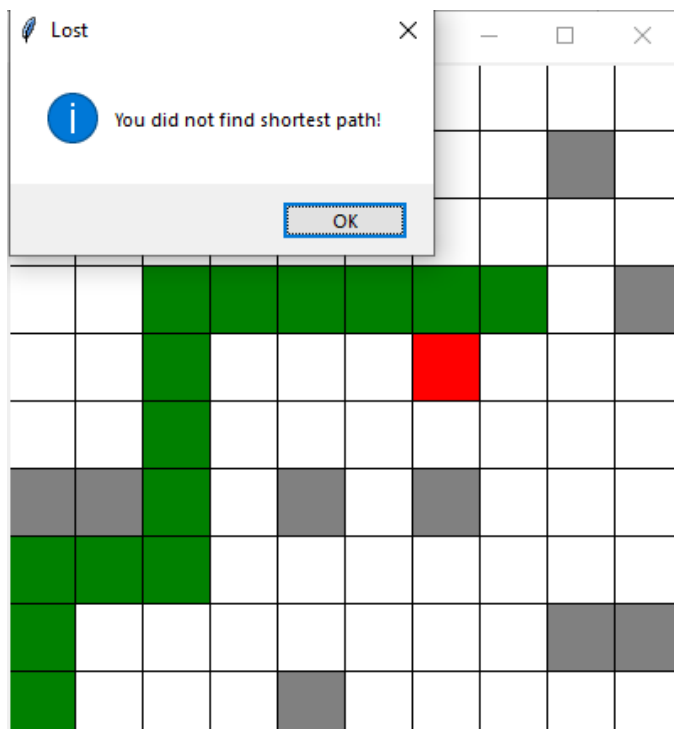
Upon hitting spacebar, the algorithm calculates and marks the correct shortest path. If it matches then you win.



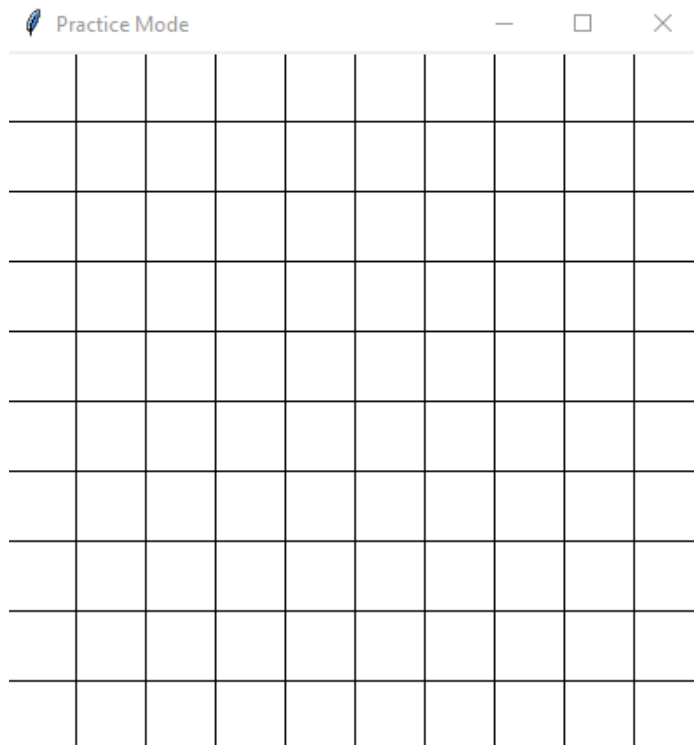
If it does not match, the tries counter decreases.



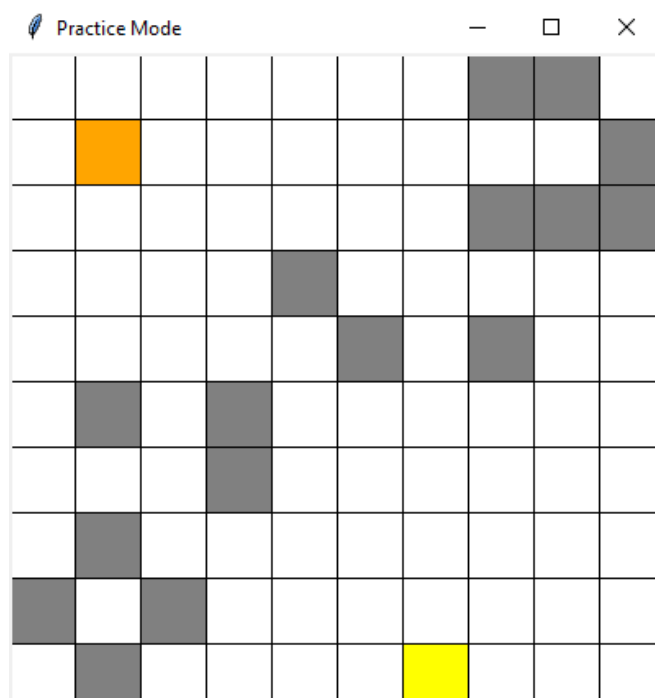
Game finishes when the tries counter runs out and the answer is shown to the user in green.



In practice mode, user is presented with an empty grid.



Here the user can mark starting and ending points, and obstacles according to the user's preference.



Upon hitting spacebar the algorithm calculates the shortest path for the user.

